

Detecting and Neutralizing Data Poisoning in Training Pipelines

An attack and defense pipeline for label flipping and backdoor trigger poisoning on CIFAR 10

GISEC School of Cybersecurity Challenge, Data Poisoning Detection Pipeline

Team

Jannat · Neelima · Shraddha

Code repository: github.com/neelima0078/gisec

Dataset: CIFAR 10, 10,000 image subset · Framework: PyTorch · Environment: Google Colab (GPU)

1. Project Objective

A model is only as trustworthy as the data it was trained on. Anyone who can influence even a small fraction of a training set, whether through a crowdsourced labeling pipeline, a scraped dataset, or a stream of user submitted content used for fine tuning, can plant behavior that survives training and activates only when the attacker chooses. The model can pass every normal accuracy check and still be compromised.

Our objective was to build a working pipeline that first demonstrates this risk concretely by injecting two distinct types of poisoning attack into a real dataset, then proves those attacks succeed while remaining invisible to standard accuracy testing, then detects the poisoned samples before training completes using more than one independent technique, and finally shows that removing the flagged samples and retraining actually eliminates the attack rather than just hiding it.

Who this is for

The realistic deployment target is any organization training or fine tuning a model on data it does not fully control. Examples include a customer support platform automatically tagging tickets submitted by users, a content moderation model retrained on flagged submissions, or a company incorporating crowdsourced or scraped data into an internal fine tuning loop. In these settings, a single malicious or careless data contributor can poison future training runs without ever touching the model itself. Today this risk is usually handled either with no data validation at all, or with slow manual spot checks by a data science team before a training run is approved. This pipeline is designed to sit in that gap, acting as an automated check that a dataset must pass before it is trusted, similar in spirit to a security scan that code must pass before it can be merged.

Why this approach, and why now

Data poisoning defenses based on manually reviewing every sample do not scale to the volume of data now used to fine tune models, much of which is scraped, crowdsourced, or continuously collected from live user feedback loops. This is a live and growing attack surface, not a solved problem from an earlier era of smaller, manually curated datasets. Loss based outlier detection and activation space clustering are both current, actively studied techniques for this exact problem, because they work directly on what the model itself learned, rather than requiring a human to inspect data by hand or trust a data source's reputation.

2. Proposed Solution

Baseline and attacks

We trained a small CNN on a 10,000 image CIFAR 10 subset as a clean baseline, then injected two attacks at a 5% poison rate, within the 1 to 10% range specified in the brief, each targeting a different weakness:

- **Label flipping.** Five percent of one class's images, airplane, had their labels silently changed to another class, automobile, with pixels left untouched. This tests whether a model can be corrupted through dishonest labeling alone.
- **Backdoor trigger.** A small three by three pixel patch was stamped onto 5% of images, drawn from any class, and relabeled to a single target class, bird. This teaches the model a shortcut: recognize the patch, ignore the actual content, output the target class on command.

Both attacks were confirmed stealthy. Models trained on poisoned data scored within one to two points of the 60.05% clean baseline on standard test accuracy, meaning a normal evaluation process would not flag anything unusual.

Detection: two independent techniques

We deliberately paired detectors that target different attack signatures, since neither one is well suited to both attacks:

- **Loss based outlier detection.** A label flipped image forces the model to memorize a label that contradicts its own learned features, producing unusually high training loss on that sample. We rank samples by per example loss and flag the top 15%, the 85th percentile, a threshold chosen by sweeping the 80th to 97th percentile range and picking the point where recall and false positive rate trade off most reasonably.
- **Activation clustering.** A backdoor trigger is a strong, consistent visual signal, so triggered images tend to form their own tight cluster in the model's internal feature space, distinct from genuine examples of the target class. We extract the last hidden layer features for the target class, split it into two clusters with k means, and flag the smaller cluster as suspicious.

Innovation: quarantine and forensic evidence layer

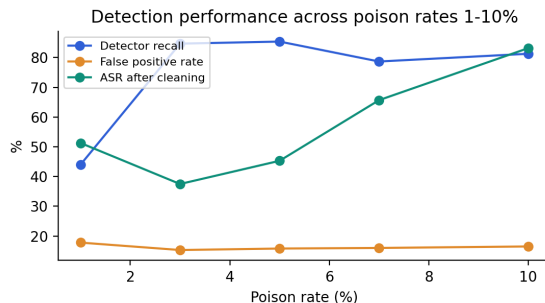
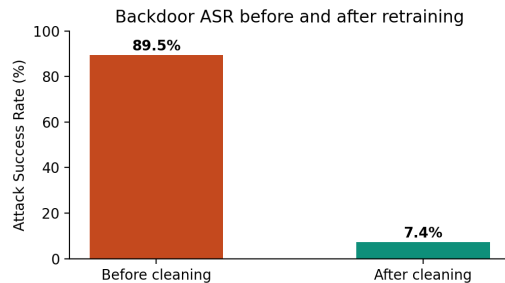
Rather than only reporting a recall percentage, we built a response layer on top of the two validated detectors. Every training sample is sorted into a **clean set**, used for retraining, or a **quarantine set**, removed but preserved rather than discarded. Each quarantined sample carries a record of exactly which detector or detectors flagged it and why. Being flagged independently by both the loss based and activation clustering detectors is a stronger signal than a single flag, and this distinction is preserved rather than collapsed into one number. This turns a bare metric into an auditable trail that a security or data science team could actually review before permanently discarding data, which matters because a detector will never be perfect and false positives have a real cost.

3. Solution Validation

Core results

Metric	Value
Clean baseline accuracy	60.05%
Poisoned model accuracy (label flip / backdoor)	61.81% / 60.40%
Attack Success Rate (ASR) before cleaning	89.48%
Detector 1 (loss based) recall / false positive rate	72.9% / 14.7%
Detector 2 (activation clustering) recall / false positive rate	79.0% / 0.0%
Post cleaning accuracy	59.09%
Post cleaning ASR	7.37%

Both poisoned models stayed within two points of the clean baseline, confirming the attacks are stealthy under normal testing. After quarantining flagged samples and retraining from scratch, not fine tuning, ASR collapsed from 89.48% to 7.37%, close to the roughly 10% random chance floor for a 10 class problem, while accuracy held within one point of baseline despite removing roughly 18.5% of the training set.



Stress testing across the required 1 to 10% poison rate range

The brief specifies a realistic 1 to 10% poison rate, so we re ran the full backdoor attack, detection, and cleaning cycle at five rates rather than tuning to a single convenient value. Detection is weakest at 1%, 44.0% recall, because too few poisoned samples exist for k means to isolate a stable minority cluster. More importantly, at 10% poison rate, post cleaning ASR only drops to 83.2%, meaning the defense largely fails there. This happens because activation clustering assumes poisoned samples are a minority within the target class; at 10% poison rate the target class becomes roughly half poisoned and half genuine, and the minority cluster assumption breaks down. We consider this an honest and expected limitation rather than a flaw to hide, and return to it in Section 4.

Reproducibility

The full pipeline was run twice, independently, from a cold runtime restart. Results were consistent across runs, with ASR before cleaning at 89.61% and 89.48%, and ASR after cleaning at 9.56% and 7.37%, confirming the pipeline's behavior is not a one off artifact of a lucky random seed.

4. Results and Conclusions

The pipeline demonstrates the full attack and defense cycle the brief asks for: two attacks that succeed while remaining statistically invisible on standard accuracy tests, two independent detection techniques that catch a majority of poisoned samples with a manageable false positive cost, and a retraining step that measurably eliminates the attack's effect rather than merely reducing it.

What we consider genuinely novel here

Most detect and remove pipelines report a single pair of numbers, recall and false positive rate, and stop there. We added a quarantine and forensics layer that preserves removed samples along with an explanation of why each was flagged, rather than silently discarding them. This is a small addition on top of detectors we had already validated, but it changes the pipeline from a one shot filter into something closer to an auditable security response, the kind of artifact a data science or security team could actually inspect and act on, rather than a black box accept or reject decision.

Limitations, stated plainly

- Activation clustering's minority cluster assumption breaks down as poison rate approaches 10%, where poisoned samples stop being a clear minority within the target class. We measured this directly rather than assuming our detectors generalize across the full specified range.
- We also attempted an unsupervised version of both detectors, meaning no ground truth poison labels used to pick thresholds, which is the realistic production setting, using Gaussian mixture and cluster ratio heuristics. Early results were inconsistent enough that we chose not to include them as a claimed result rather than present numbers we could not stand behind. This remains a natural next step.
- Our detectors were built and tuned against a specific backdoor trigger, a fixed patch in a fixed position. A trigger that varies in position, size, or color across poisoned samples would likely be harder for the activation clustering detector to isolate cleanly.

Conclusion

The core claim we can defend with evidence, not just assertion, is this: a small, realistic fraction of poisoned training data can silently compromise a model while passing every standard accuracy check, and a two technique detection and retraining pipeline can substantially neutralize that compromise across most of the specified poison rate range, with a known and measured boundary condition at the high end. Full code, the notebook with all outputs, and setup instructions are available in the repository linked on the title page.